

Data Wrangling with Stata

Workshop 2 · 75 minutes · Five operations: collapse, append, merge, reshape (wide and long)

Setting up

Double-click `start_here.do` inside the workshop folder. Stata opens with its working directory already set to that folder. Run the two lines it contains:

```
pwd
dir *.dta
```

`pwd` should print the workshop folder. `dir *.dta` should list 14 files. If both look right, you are set for the whole session and you will never type a file path again.

If `pwd` shows the wrong folder, Stata was already open and kept its old working directory. Fix it with `cd`: open the folder in File Explorer, click the address bar, copy the path, then type `cd "` → paste → close the quote. `start_here.do` has the full instructions and an example path.

How this sheet works

Shaded lines are the ones you work out. A grey background means the line has slots in it — (`statistic`), `variable(s)` and so on — and the menu just below lists what goes in them. Working out which is the exercise. **Lines with no shading are written out in full:** type those as they appear. `use`, `save` and `reshape` are always unshaded.

Two kinds of file names. The files provided to you are named `block1_1980`, `block3a` and so on — the number tells you which block to open it for. The files *you* make get descriptive names: `state_1980`, `state_race`. So your work never overwrites the files you might need to fall back on.

If a block does not work out, open the next block's file and carry on. That is what they are for. You will not be behind. *If in any doubt, open the provided file.*

Every block ends with `describe`, and this sheet prints the number of observations and variables you should see. That is how you check yourself without asking.

One note on the numbers. Population counts in these files are already weighted estimates — the survey weight was applied when the data was built. There is no weight for you to add.

Block 1 Collapse aggregating the data

390 rows of counties become 51 rows of states.

```
use block1_1980, clear
describe, short
```

```
collapse (statistic) variable(s), by(key variable(s))
```

```
describe
save state_1980, replace
```

<i>statistic</i>	sum	mean	firstnm	min	max
<i>variable(s)</i>	pop	white	black	hispanic	asian
<i>key variable(s)</i>	year	statefip			

Check: 51 observations, 9 variables. `fips5` is gone — `collapse` keeps only the `by()` variables and the ones you named a statistic for. Everything else is dropped.

Second pass, if there is time — more than one statistic in a single command. Do not save over your working file.

```
collapse (statistic) variable(s) (statistic) newname = variable, by(key(s))
```

Finished early? Collapse to national totals instead, with `by(year)` alone. Or open `block4` — a file from later in the session — and collapse it to one row per state across all three decades. **Do not save over your working file**, and do not save over `block4` either: Blocks 3, 4 and 5 all still need it.

Block 2 Append stacking observations

Three years, three separate files, one dataset. 51 rows become 153.

```
use block2_1980, clear
```

```
append using filename
append using filename
```

```
describe
save state_race, replace
```

```
filename    block2_1990    block2_2000
```

Check: 153 observations, 9 variables. The variable count did not change. `append` adds rows, not columns — and it only works because all three files use the same variable names and types.

Second pass, if there is time — both files in one command. Not saved.

```
append using filename filename
```

Finished early? Build the stack with no `use` at all: start from `clear` and append all three. Or write it as a `foreach` loop — use the do-file editor for that, not the command window.

Block 3 Merge joining variables on a key

Race/ethnicity in one file, age groups in another, matched row to row.

```
use block3a, clear
```

```
isid key variable(s)
```

```
merge type key variable(s) using filename
```

```
tab _merge
drop _merge
```

```
describe
save state_long, replace
```

```
type                    1:1    m:1    1:m
key variable(s)        year statefip
filename                block3b
_merge                 1 = master only    2 = using only    3 = matched both
```

`isid` asks whether those variables really identify a row uniquely — *is* this an *id*? It says nothing at all when the answer is yes, and stops you when it is no. Run before a merge, it is a cheap way to find out that your key is not a key — at the point where you can still do something about it.

Read `_merge` before you drop it. It is the only evidence that the merge did what you meant. All 153 rows should come back as 3. Rows of 1 or 2 mean a key did not line up, and that is a problem to solve, not a number to skim past. When a merge that obviously should work comes back unmatched, the usual cause is the key types — see **the last section of this sheet**.

`_merge` is sometimes worth keeping — to build a flag from, say. If you keep it, rename it as it is created, with `merge`'s `generate()` option, rather than leaving the default name for your next merge to collide with. Here, check it, then drop it.

pop is in both files. Both copies hold the same values, so nothing is lost: `merge` keeps the master's copy and discards the other. When the values *differ*, that choice starts to matter — see the last suggestion below.

Check: 153 observations, 13 variables. If you see 14, `_merge` is still there.

Second pass, if there is time — a many-to-one merge, attaching each state's Census region from a crosswalk. Not saved.

```
use block3a, clear
```

```
merge m:1 statefip using xwalk_statefip_region
```

Finished early? Run the same join from the other side — crosswalk as master, 1:m — and watch only the direction change. (`joinby` handles many-to-many, which `merge` refuses.) Try `nogenerate`, or `generate()` with a name of your own. Or run a deliberately *wrong* merge to see what `update` and `replace` do: `block2_1980` as master against `block3b_1990`, keyed on `statefip` alone, so 1980 rows are joined to 1990 ones. Both files carry `pop` with genuinely different values. Compare plain `merge`, then `,` `update`, then `,` `update replace` — and notice that `year` gets overwritten too, silently relabelling every 1980 row as 1990.

Block 4 Reshape long → wide

One row per state-year becomes one row per state, with a column per year. This is the least guessable syntax in Stata, so it is written out in full.

```
use block4, clear

isid statefip year

reshape wide pop white black hispanic asian native mix_other ///
          age_prek age_minor age_working age_senior, i(statefip) j(year)

describe
save state_wide, replace

i()    the variable that identifies a row in the result
j()    the variable whose values become part of the new column names
```

`isid` is worth running before a `reshape` — the same check as in Block 3, for the same reason. `reshape` needs `i()` to identify a row uniquely, and a failed `isid` is much easier to read than a failed `reshape`.

Check: 51 observations, 34 variables. 13 variables became 34: `statefip`, plus each of the other 11 once per year ($11 \times 3 = 33$). `pop` is now `pop1980`, `pop1990`, `pop2000`.

Finished early? Reshape on state instead of year — `i(year) j(statefip)`. Nothing is wrong with that command; it just answers a different question, and the answer has 3 rows and roughly 560 columns. Shape follows your choice of `i()` and `j()`, not the data. Do not save over your working file.

Block 5 Reshape wide → long

The reverse. Compare the two commands — the varlist and the options are identical, and only the direction word changes.

```
use block5, clear

isid statefip

reshape long pop white black hispanic asian native mix_other ///
          age_prek age_minor age_working age_senior, i(statefip) j(year)

describe

Block 4  reshape wide (same varlist) , i(statefip) j(year)
Block 5  reshape long  (same varlist) , i(statefip) j(year)
```

Check: 153 observations, 13 variables. You are back to the shape you had at the end of Block 3 — which is the point. Wide and long hold the same data.

Finished early? Try `age*` in place of the four age variables. It will fail, and the failure is worth seeing: `reshape wide` takes a *varlist*, so `age*` expands normally there, but `reshape long` takes *stubnames* — the part of the name left after the year is stripped off, and an asterisk is not legal in a name. You should get exactly:

```
* invalid name
r(198);
```

A different error means a different mistake.

One last thing `destring` and `tostring`

Merges match on keys, and keys have to be the same type in both files. FIPS codes are identifiers, not quantities: stored as a number, California's 06 becomes 6 and the leading zero is gone for good. A string key will never match a numeric one.

```
tostring statefip, generate(statefip_str)
replace statefip_str = "0" + statefip_str if strlen(statefip_str) == 1
```

`tostring` changes the type, and that is only half the job: it gives you "6", not "06". The second line puts the zero back. Padding is the part that actually makes a FIPS key match. Going the other way, `destring` converts text back to a number — and drops the leading zero again, which is exactly the trap.

```
destring statefip_str, replace
```

Today's files are already consistent, so you do not need any of this. It is the first thing to check when a merge that should obviously work does not.